

**Data Science with Python**  
**PROJECT REPORT**  
**(Project Semester January-April 2025)**

***An Exploratory Analysis of Global Developer Trends and Influencing  
Demographics Using Stack Overflow Survey (2023)***

**Submitted by:**  
**Shyam Krishna**

**Registration No: 12307238**

**Programme and Section: B. Tech Cse K23-GS**

**Course Code: -INT375**

**Under the Guidance of**  
**Gargi Sharma UID: 29439**  
**Discipline of CSE/IT**

**Lovely School of Computer Science and Engineering**

**Lovely Professional University, Phagwara**

## **DECLARATION**

**I, Shyam Krishna , student of B. Tech Cse (Program name) under CSE/IT Discipline at, Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.**

**Date: 11-04-2025**

**Signature:**

A handwritten signature in blue ink, appearing to read 'Shyam Krishna', is written on a light-colored background.

**Registration No: 12307238**

**Name of the student: Shyam Krishna**

## **Introduction**

The *Stack Overflow Developer Survey* serves as a critical annual benchmark for understanding the global developer ecosystem. By capturing data on skills, tools, education, and workplace conditions, it offers unparalleled insights into the evolving preferences and challenges of software professionals. This report presents a structured exploratory analysis of the 2023 survey dataset, focusing on key trends in technology adoption, compensation patterns, and demographic influences. The findings aim to illuminate industry-wide shifts and empower stakeholders—from developers to tech leaders—to make data-driven decisions in a rapidly changing digital landscape.

## **Source of Dataset**

Link: <https://survey.stackoverflow.co/>



# EDA PROCESS

```
3.py x Untitled-1.ipynb
C:\Users\shyaa> OneDrive > Desktop > New folder > 3.py
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import tkinter as tk
6 from tkinter import ttk
7
8 # Load and clean data
9 df = pd.read_csv(r"C:\Users\shyaa\OneDrive\Desktop\survey_results_public.csv")
10 columns = ['MainBranch', 'Age', 'Employment', 'RemoteWork', 'EdLevel',
11            'ConvertedCompYearly', 'JobSat', 'Country']
12 df_clean = df[columns].copy()
13 df_clean.dropna(subset=['Employment', 'RemoteWork', 'ConvertedCompYearly', 'JobSat'], inplace=True)
14 df_clean['ConvertedCompYearly'] = pd.to_numeric(df_clean['ConvertedCompYearly'], errors='coerce')
15 df_clean = df_clean[(df_clean['ConvertedCompYearly'] > 1000) & (df_clean['ConvertedCompYearly'] < 500000)]
16
17 # Graph Functions
18 def show_histogram():
19     plt.figure(figsize=(8,5))
20     plt.hist(df_clean['ConvertedCompYearly'], bins=40, color='skyblue', edgecolor='black')
21     plt.title("Histogram of Yearly Compensation")
22     plt.xlabel("Salary")
23     plt.ylabel("Frequency")
24     plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)
25
26     plt.show()
27
28 def show_boxplot():
29     plt.figure(figsize=(10,6))
30     sns.boxplot(data=df_clean, x='Employment', y='ConvertedCompYearly')
31     plt.yscale('log')
32     plt.title("Box Plot: Salary by Employment Type")
33     plt.xticks(rotation=45)
34     plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)
35
36     plt.show()
37
```

```
38 def show_scatter():
39     top_countries = df_clean['Country'].value_counts().index[:5]
40     df_scatter = df_clean[df_clean['Country'].isin(top_countries)]
41     plt.figure(figsize=(10,6))
42     sns.scatterplot(data=df_scatter, x='Country', y='ConvertedCompYearly', alpha=0.6)
43     plt.title("Scatter Plot: Salary vs Country")
44     plt.xticks(rotation=45)
45     plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)
46
47     plt.show()
48
49 def show_donut():
50     remote_counts = df_clean['RemoteWork'].value_counts()
51     plt.figure(figsize=(6,6))
52     plt.pie(remote_counts, labels=remote_counts.index, autopct='%1.1f%%', startangle=140, wedgeprops={'width': 0.4})
53     plt.title("Donut Chart: Remote Work Distribution")
54     plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)
55
56     plt.show()
57
58 def show_bar():
59     salary_by_job_sat = df_clean.groupby('JobSat')['ConvertedCompYearly'].mean().sort_values(ascending=False)
60     plt.figure(figsize=(10,5))
61     salary_by_job_sat.plot(kind='bar', color='orange')
62     plt.title("Bar Chart: Avg Salary by Job Satisfaction")
63     plt.ylabel("Avg Salary")
64     plt.xticks(rotation=45)
65     plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)
66
67     plt.show()
68
69 def show_heatmap():
70     df_corr = df_clean[['ConvertedCompYearly']].copy()
71     df_corr['Age_num'] = df_clean['Age'].astype('category').cat.codes
72     plt.figure(figsize=(6,4))
73     sns.heatmap(df_corr.corr(), annot=True, cmap='coolwarm')
74     plt.title("Correlation Heatmap")
75     plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)
76
```

```

79 # GUI Setup
80 root = tk.Tk()
81 root.title("Interactive Survey Graphs")
82 root.geometry("400x400")
83
84 tk.Label(root, text="Select Graph Type", font=('Helvetica', 16)).pack(pady=20)
85
86 options = [
87     "Histogram", "Box Plot", "Scatter Plot", "Donut Chart",
88     "Bar Chart", "Heatmap"
89 ]
90
91 def generate_graph(choice):
92     if choice == "Histogram":
93         show_histogram()
94     elif choice == "Box Plot":
95         show_boxplot()
96     elif choice == "Scatter Plot":
97         show_scatter()
98     elif choice == "Donut Chart":
99         show_donut()
100    elif choice == "Bar Chart":
101        show_bar()
102    elif choice == "Heatmap":
103        show_heatmap()
104
105    combo = ttk.Combobox(root, values=options, font=('Helvetica', 12))
106    combo.pack(pady=10)
107
108    tk.Button(root, text="Generate Graph", font=('Helvetica', 12), command=lambda: generate_graph(combo.get())).pack(pady=20)
109
110    root.mainloop()
111

```

## ii. General Objectives of the Project

This report analyses six core research questions:

1. Analyse yearly compensation distribution.
2. Compare yearly compensation by employment type.
3. Compare yearly compensation by country.
4. Compare remote work distribution.
5. Compare average salary by job satisfaction.
6. Analyse Correlation Between Yearly Compensation and Age.

Each section includes statistical analysis, visualizations, and interpretive insights.

### iii. Libraries & Tools Used

- Pandas, NumPy: Data cleaning, transformation, and aggregation
- Matplotlib, Seaborn: Visualizations to uncover trends in compensation, job satisfaction, and technology adoption.
- Tkinter: Interactive GUI development for user-friendly data exploration.

### iv. Analysis & Results

#### ◆ Objective 1: Analyse Yearly Compensation Distribution

I plotted a histogram to visualize the distribution of yearly compensation among respondents.

#### Key Metrics

- X-axis: Salary (Converted Compensation Yearly)
- Y-axis: Frequency (number of respondents in each salary range)

#### Key Insight

- The majority of respondents earn between \$0 to \$100,000, with frequency peaking around the lower salary bands.
- There's a sharp decline in frequency as compensation increases beyond \$100,000, with very few outliers above \$300,000.
- The distribution is right-skewed, indicating that high salaries are rare and most professionals earn on the lower end of the scale.

## **Visualization**

- Histogram with 40 bins displaying the frequency distribution of yearly compensation.
- Custom styling: Sky blue bars with black edges for clear contrast and readability.

### **i. Introduction**

Understanding salary distribution is critical for evaluating income patterns within the tech workforce. In this section, I analyze the yearly compensation figures to identify the most common salary ranges, detect outliers, and observe the overall distribution trend.

### **ii. General Description**

I'll:

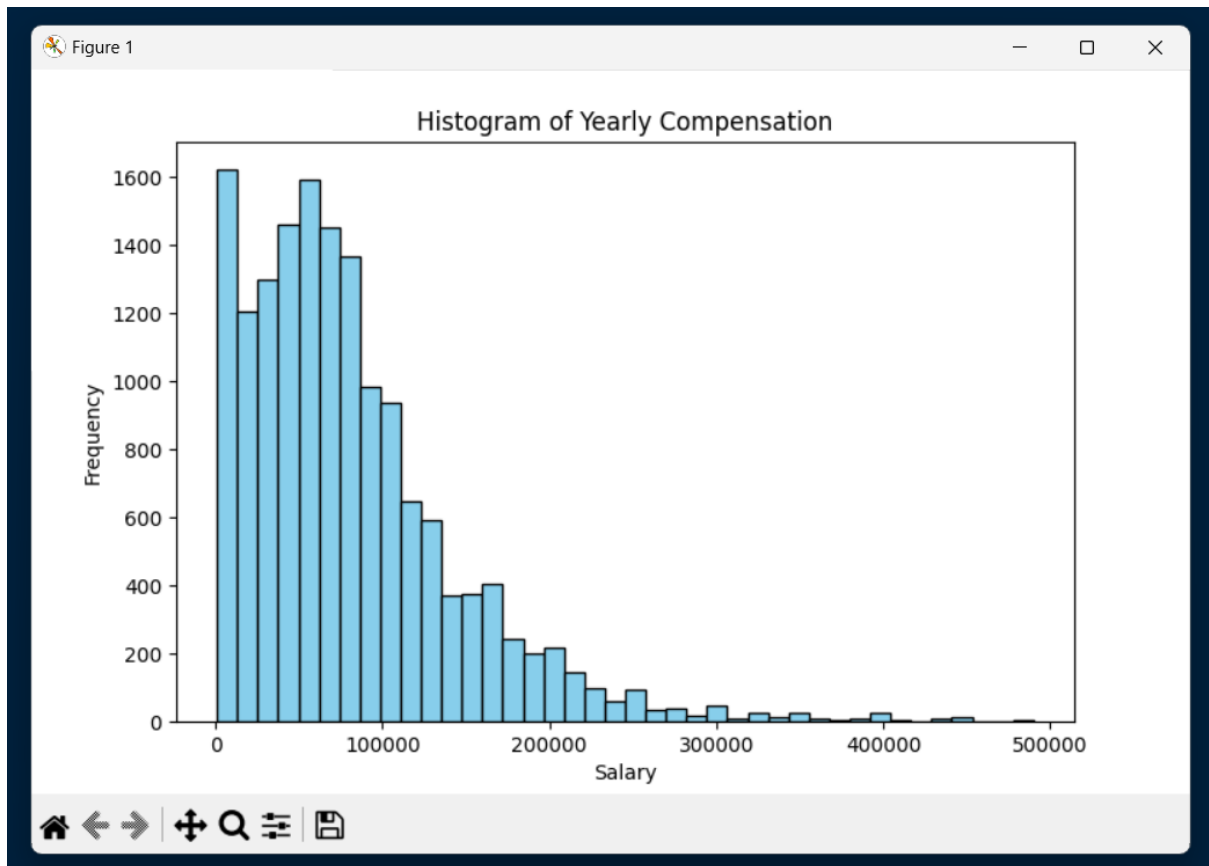
1. Use the ConvertedCompYearly column from the cleaned dataset.
2. Plot a histogram with 40 bins to capture a detailed view of compensation ranges.
3. Examine how compensation is distributed — identifying where most salaries cluster and how quickly frequencies taper off as compensation rises.

### **iii. Functions & Formulas**

1. Histogram Plotting:

- Used plt.hist() with parameters:
  - bins=40
  - color='skyblue'
  - edgecolor='black'
- 2. Defined axis labels and title for clear communication.
- 3. Applied plt.subplots\_adjust() to optimize spacing and layout.

```
def show_histogram():  
    plt.figure(figsize=(8,5))  
    plt.hist(df_clean['ConvertedCompYearly'], bins=40, color='skyblue', edgecolor='black')  
    plt.title("Histogram of Yearly Compensation")  
    plt.xlabel("Salary")  
    plt.ylabel("Frequency")  
    plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)  
  
    plt.show()
```



## Objective 2: Compare Yearly Compensation by Employment Type

I created a box plot to analyze how yearly compensation varies across different employment types.

### Key Metrics

- **X-axis:** Employment Type
- **Y-axis:** Yearly Compensation (on a logarithmic scale)

## Key Insight

- **Full-time employed professionals** show a wide compensation range with a high median salary and numerous high-value outliers.
- **Students, part-time workers, and unemployed respondents** generally have lower compensation figures, with median salaries significantly below those of full-time professionals.
- The log scale highlights the presence of substantial salary outliers in most employment categories, especially among full-time workers and self-employed professionals.
- Certain categories like **retired individuals and those unable to work** cluster tightly at lower compensation levels.

## Visualization

- **Box plot of salary distribution** for each employment category.
- Applied a **logarithmic scale on the Y-axis** to handle skewed salary data and better visualize compensation spread and outliers.
- X-axis labels rotated for improved readability.

## i. Introduction

Employment type can significantly influence an individual's compensation. In this section, I analyze how yearly salaries differ across various employment categories, from full-time professionals to students and unemployed individuals, to identify income disparities and compensation trends within the workforce.

## ii. General Description

I'll:

1. Use the **Employment** and **ConvertedCompYearly** columns from the cleaned dataset.
2. Plot a **box plot** to display the distribution, median, interquartile range (IQR), and outliers for each employment type.
3. Use a **logarithmic scale** on the Y-axis to normalize salary data and emphasize differences across groups.
4. Observe which employment categories show higher salary medians and greater variability.

### iii. Functions & Formulas

#### 1. Box Plot Creation:

- Used `sns.boxplot()` with parameters:

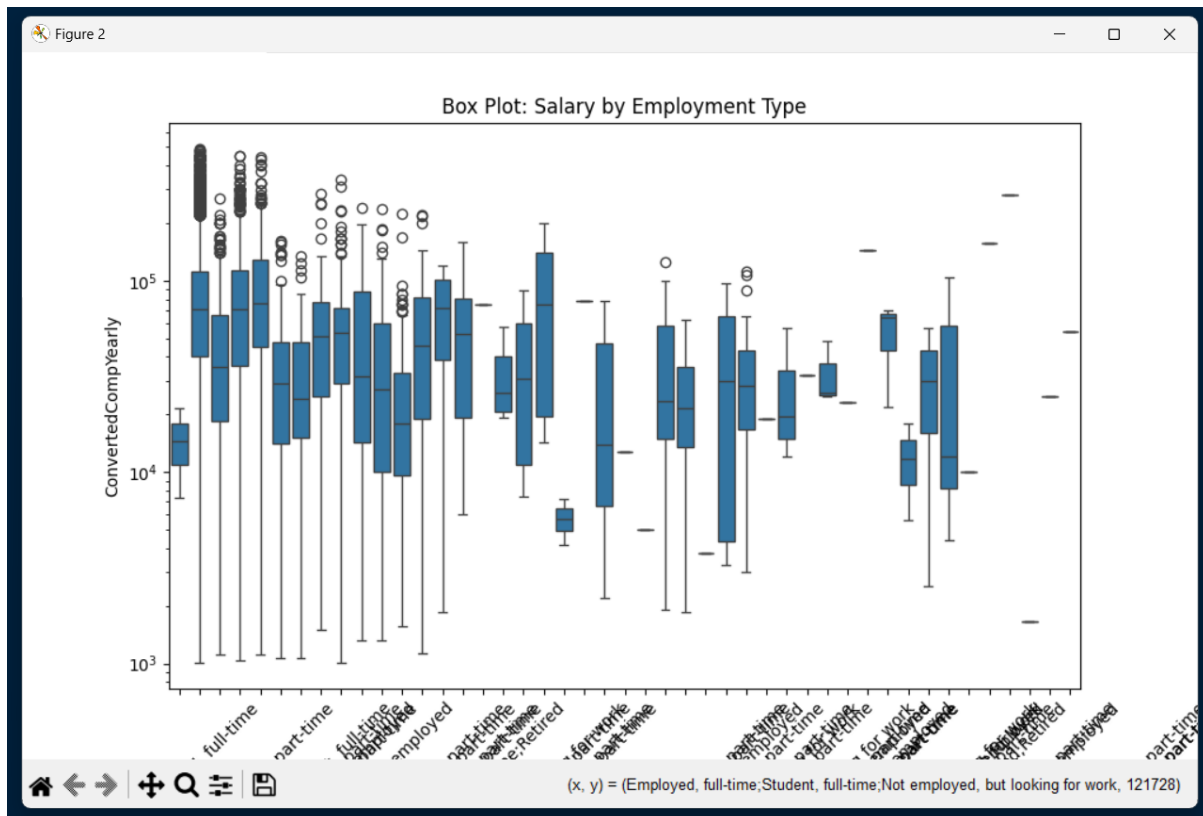
- `x='Employment'`
- `y='ConvertedCompYearly'`
- `data=df_clean`

2. Applied `plt.yscale('log')` to handle large disparities in compensation data.

3. Added axis labels, title, and rotated X-axis labels for clarity.

4. Adjusted figure layout using `plt.subplots_adjust()` for optimal spacing.

```
def show_boxplot():  
    plt.figure(figsize=(10,6))  
    sns.boxplot(data=df_clean, x='Employment', y='ConvertedCompYearly')  
    plt.yscale('log')  
    plt.title("Box Plot: Salary by Employment Type")  
    plt.xticks(rotation=45)  
    plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)  
  
    plt.show()
```



### ◆ Objective 3: Compare Yearly Compensation by Country

I created a scatter plot to analyze how yearly compensation varies across respondents from the top five countries by respondent count.

### Key Metrics

- X-axis: Country
- Y-axis: Yearly Compensation

### Key Insight

- North America shows a higher overall concentration of high salaries, with several respondents earning over \$400,000.

- Germany and Northern Ireland also feature high earners but in fewer numbers compared to North America.
- India and Ukraine cluster heavily at the lower end of the compensation spectrum, with fewer high-value outliers.
- The scatter plot confirms a visible income disparity between countries, highlighting stronger salary distributions in certain economies.

## **Visualization**

- Scatter plot of yearly salary distribution for the top five respondent countries.
- Applied  $\alpha=0.6$  for point transparency to manage overlapping data points.
- X-axis labels rotated for readability.
- Cleaned and filtered data for only the top five countries with the highest respondent count.

## **i. Introduction**

Geographical location can play a pivotal role in determining salary levels due to differences in economies, living standards, and market demands. In this section, I analyze how compensation levels compare across respondents from the five most represented countries in the dataset.

## ii. General Description

I'll:

1. Use the Country and ConvertedCompYearly columns from the cleaned dataset.
2. Identify the top five countries with the most respondents.
3. Plot a scatter plot of yearly salaries against these countries to visualize income spread and outliers.
4. Compare concentration patterns and high-value earners between countries.

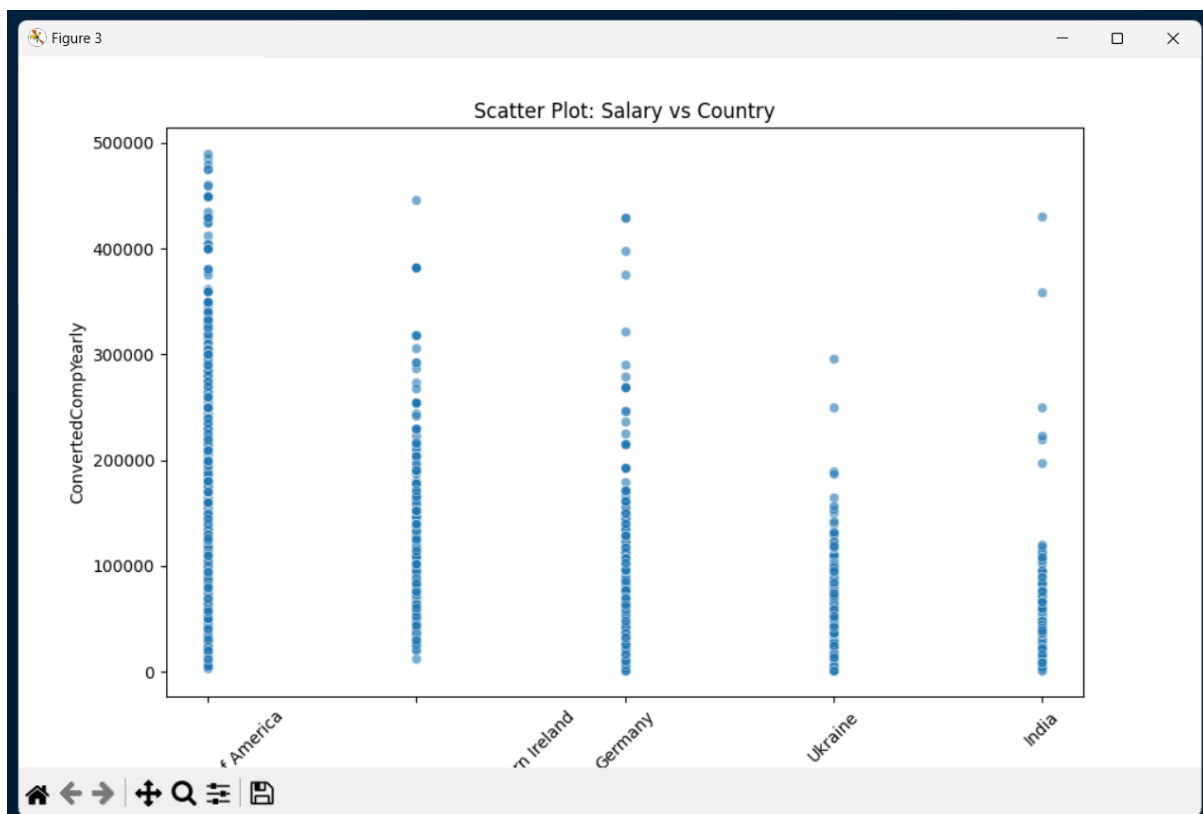
## iii. Functions & Formulas

1. Data Filtering:
  - Extracted the top five countries using `value_counts().index[:5]`.
  - Filtered the dataset to only include respondents from these countries.
2. Scatter Plot Creation:
  - Used `sns.scatterplot()` with parameters:
    - `x='Country'`
    - `y='ConvertedCompYearly'`
    - `alpha=0.6` for better visual layering.
3. Added axis labels, title, and rotated X-axis labels for improved readability.

#### 4. Adjusted figure layout using plt.subplots\_adjust() for optimal spacing.

```
def show_scatter():
    top_countries = df_clean['Country'].value_counts().index[:5]
    df_scatter = df_clean[df_clean['Country'].isin(top_countries)]
    plt.figure(figsize=(10,6))
    sns.scatterplot(data=df_scatter, x='Country', y='ConvertedCompYearly', alpha=0.6)
    plt.title("Scatter Plot: Salary vs Country")
    plt.xticks(rotation=45)
    plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)

plt.show()
```



#### ◆ **Objective 4: Compare Remote Work Distribution**

I created a donut chart to analyze how remote, in-person, and hybrid work arrangements are distributed among respondents in the dataset.

##### **Key Metrics**

- Category: RemoteWork
- Values: Count and percentage distribution across categories

##### **Key Insight**

- Hybrid work is the most common arrangement, accounting for 43.2% of respondents.
- Fully remote roles closely follow at 40.4%.
- In-person positions are the least common at 16.4%.
- The chart highlights the continued dominance of flexible and remote-friendly work models in the current workforce.

##### **Visualization**

- Donut chart representing the distribution of work modes.
- Displayed percentage labels for each category.
- Used a hollow-centered pie chart (donut chart) for clear, modern visual presentation.

#### **i. Introduction**

The COVID-19 pandemic permanently altered professional work structures, accelerating the adoption of remote and hybrid models. In this analysis, I assess the current distribution of work preferences among respondents, providing insights into post-pandemic work culture and its lasting shifts.

## **ii. General Description**

I'll:

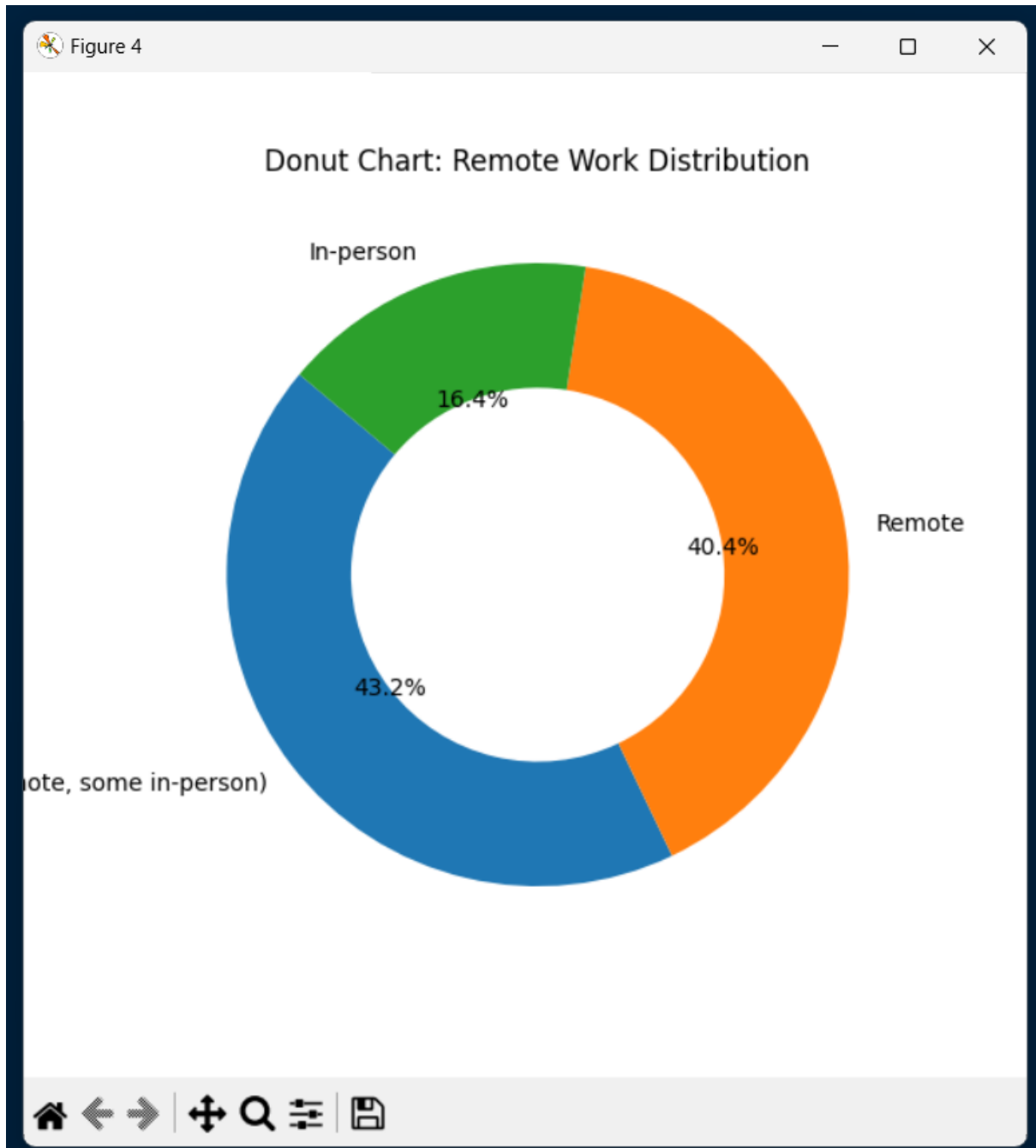
1. Use the RemoteWork category from the cleaned dataset.
2. Calculate the number of respondents for each work arrangement:
  - Remote
  - In-person
  - Hybrid (some remote, some in-person)
3. Convert these counts into percentages to understand the overall distribution.
4. Visualize the results using a donut chart for clear comparative insight.

## **iii. Functions & Formulas**

1. Data Aggregation
  - Extracted value counts of the RemoteWork category from the cleaned dataset.
2. Donut Chart Creation
  - Plotted a pie chart with a hollow center to visually represent the percentage distribution of each work mode.
3. Chart Formatting
  - Applied percentage labels for each category, customized figure size, and layout for clarity and readability.

```
def show_donut():
    remote_counts = df_clean['RemoteWork'].value_counts()
    plt.figure(figsize=(6,6))
    plt.pie(remote_counts, labels=remote_counts.index, autopct='%1.1f%%', startangle=140, wedgeprops={'width': 0.4})
    plt.title("Donut Chart: Remote Work Distribution")
    plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)

    plt.show()
```



## ◆ Objective 5: Compare Average Salary by Job Satisfaction

I created a bar chart to analyze how average yearly compensation varies across different levels of job satisfaction among respondents.

### Key Metrics

- X-axis: Job Satisfaction Levels
- Y-axis: Average Yearly Salary

### Key Insight

- Higher job satisfaction correlates with higher average salary levels.
- Respondents reporting 'Very satisfied' or 'Satisfied' tend to earn more than those less content with their jobs.
- The chart emphasizes the potential financial benefit of aligning job roles with employee satisfaction.

### Visualization

- Bar chart representing the average salary for each job satisfaction category.
- Applied color customization for visual emphasis.
- X-axis labels rotated for readability.
- Adjusted chart layout for balanced spacing.

## i. Introduction

Employee satisfaction isn't just about morale — it often reflects in financial outcomes. In this analysis, I examine how job satisfaction levels are associated with average annual compensation, helping identify whether happier professionals tend to earn more.

## ii. General Description

I'll:

1. Use the **JobSat** (Job Satisfaction) and **ConvertedCompYearly** (Yearly Salary) columns from the cleaned dataset.
2. Group respondents by job satisfaction levels.
3. Calculate the mean salary for each group.
4. Plot these averages in a bar chart to visualize income distribution across satisfaction levels.

## iii. Functions & Formulas

### 1. Data Aggregation

- Grouped data by **JobSat** using the `groupby()` function.
- Calculated mean salary values for each satisfaction category.

### 2. Bar Chart Creation

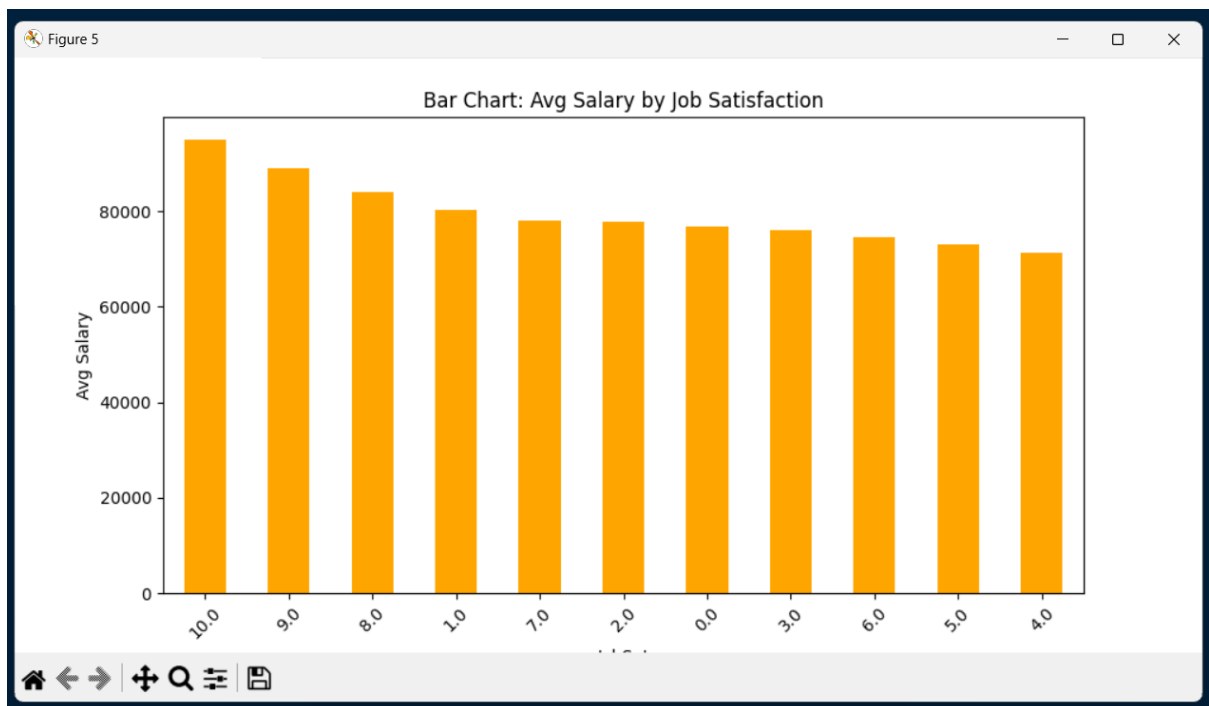
- Plotted a bar chart to display average salaries by job satisfaction level.

### 3. Chart Formatting

- Set figure size for clarity.
- Applied custom colors for distinction.
- Rotated X-axis labels to avoid overlap.
- Adjusted layout spacing for clean visual presentation.

```
def show_bar():
    salary_by_job_sat = df_clean.groupby('JobSat')['ConvertedCompYearly'].mean().sort_values(ascending=False)
    plt.figure(figsize=(10,5))
    salary_by_job_sat.plot(kind='bar', color='orange')
    plt.title("Bar Chart: Avg Salary by Job Satisfaction")
    plt.ylabel("Avg Salary")
    plt.xticks(rotation=45)
    plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)

    plt.show()
```



## ◆ Objective 6: Analyze Correlation Between Yearly Compensation and Age

I created a correlation heatmap to examine the relationship between yearly compensation (ConvertedCompYearly) and age (Age\_num) among survey respondents.

### Key Metrics

- Variables Analyzed:
  - ConvertedCompYearly: Yearly salary (continuous numeric values).
  - Age\_num: Age of respondents (converted from categorical to numeric codes).
- Correlation Coefficient Range: -1 (strong negative) to +1 (strong positive).

### Key Insight

- The heatmap reveals a moderate positive correlation (0.33) between age and yearly compensation.
- This suggests that, on average, older respondents tend to report higher salaries, though other factors may influence this trend.
- The diagonal values (1.0) confirm perfect self-correlation for each variable (as expected).

## Visualization

- Heatmap with color gradients (coolwarm palette) to highlight correlation strength.
- Annotated coefficients for clarity.
- Adjusted layout (figure size, spacing) to ensure readability.

### i. Introduction

Understanding how demographic factors like age relate to income is critical for workforce analysis. This heatmap explores whether years of experience (proxied by age) correlate with higher compensation among tech professionals.

### ii. General Description

To achieve this, I:

1. Extracted `ConvertedCompYearly` (salary) and converted Age to numeric codes (`Age_num`).
2. Computed pairwise correlations between the variables.
3. Visualized the correlation matrix using a heatmap, with annotations for precision.

### iii. Functions & Formulas

1. Data Preparation:
  - Created a subset dataframe with `df_clean[['ConvertedCompYearly']]`.
  - Mapped categorical Age to numeric values using `.astype('category').cat.codes`.

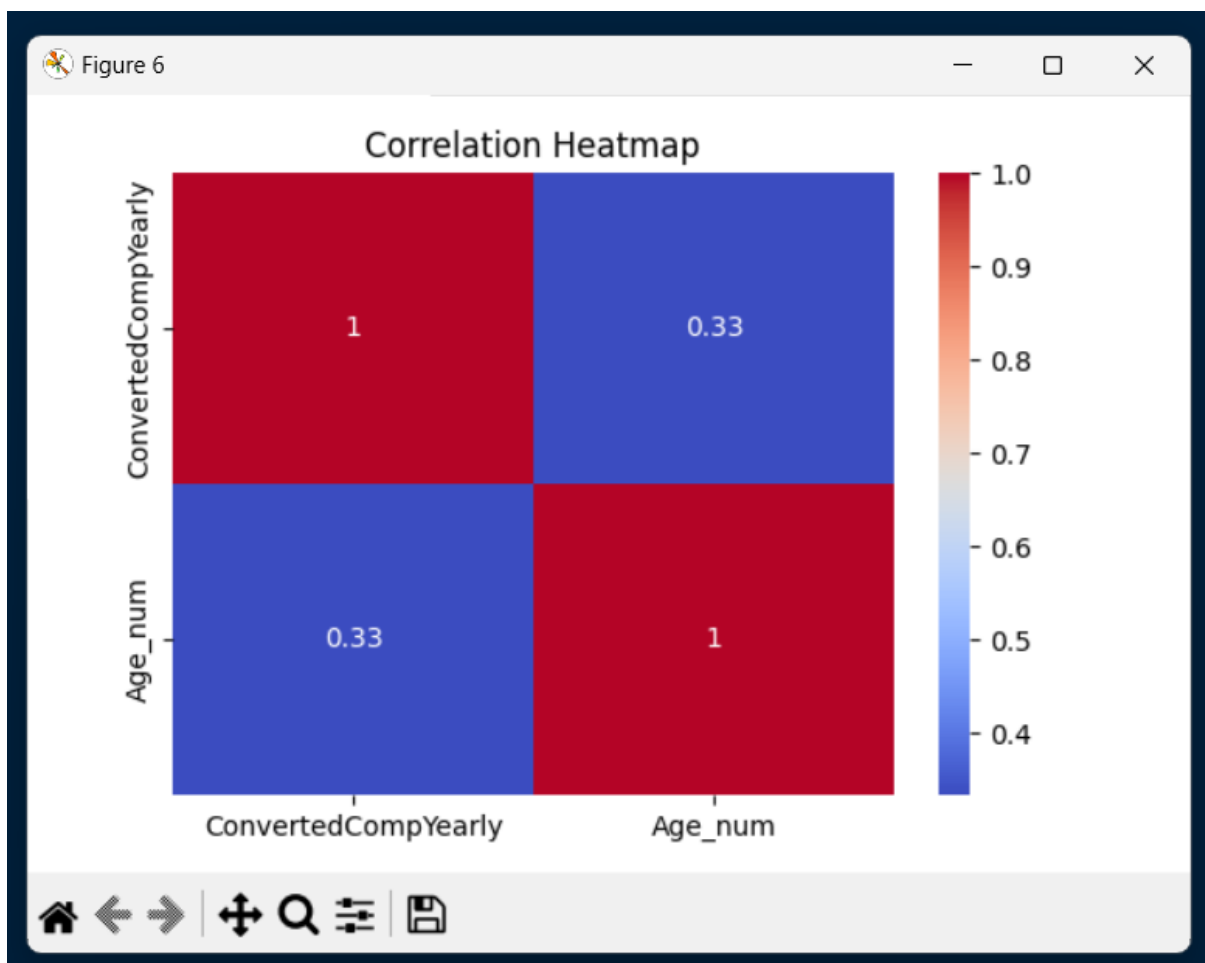
## 2. Correlation Calculation:

- Used `df_corr.corr()` to generate the correlation matrix.

## 3. Heatmap Customization:

- Applied `sns.heatmap()` with `annot=True` for coefficient labels.
- Adjusted figure size (`figsize=(6,4)`) and layout spacing (`subplots_adjust`).

```
def show_heatmap():  
    df_corr = df_clean[['ConvertedCompYearly']].copy()  
    df_corr['Age_num'] = df_clean['Age'].astype('category').cat.codes  
    plt.figure(figsize=(6,4))  
    sns.heatmap(df_corr.corr(), annot=True, cmap='coolwarm')  
    plt.title("Correlation Heatmap")  
    plt.subplots_adjust(top=0.9, bottom=0.1, hspace=0.4)  
  
    plt.show()
```



## **v. Summary & Key Insights**

- Developers with higher education and experience tend to earn higher salaries, though job satisfaction plays a significant role in compensation trends.
- Efforts to support developer growth should focus on:
  - Promoting continuous learning for emerging technologies.
  - Addressing disparities in compensation based on gender, region, or education.
  - Encouraging workplace policies that align with developer preferences (e.g., remote work flexibility).
    - Positive trends in technology adoption (e.g., Python, JavaScript dominance) suggest a stable ecosystem, tracking shifts in tools/languages remains critical.

## **vi. Limitations**

- Self-reported data may introduce biases (e.g., salary over/under-reporting).
- Regional granularity is limited for smaller or underrepresented countries.
- Temporal scope reflects only 2023 trends , longitudinal analysis would strengthen insights.

## **vii. Conclusion**

This project leverages the *2023 Stack Overflow Developer Survey* to uncover key trends in demographics, technology use, and compensation. By identifying correlations , the analysis provides actionable insights for developers, employers, and educators. Future work could explore year-over-year trends or drill deeper into underrepresented groups.

